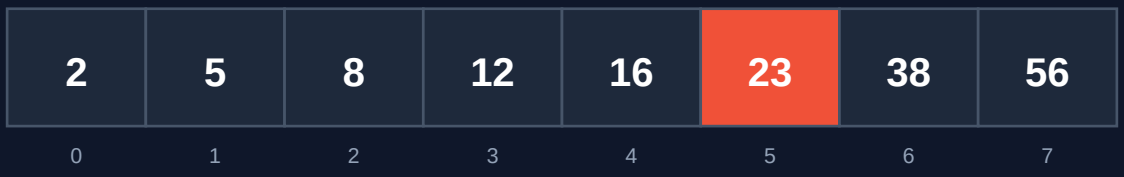
Data Structures & Algorithms in Swift

From absolute scratch: plain-English ideas, diagrams, runnable code, and a 12-week practice plan.



13 chapters | Works with Swift 5 and newer | No packages required

CONTENTS

What is inside

Before You Begin 3

The Roadmap 4

1. Swift Basics 5

2. Big-O Notation 8

3. Arrays & Strings 10

4. Hash Maps & Sets 13

5. Linked Lists 15

6. Stacks & Queues 18

7. Recursion 20

8. Sorting & Searching 22

9. Trees 24

10. Heaps & Priority Queues 26

11. Graphs 28

12. Dynamic Programming 30

13. Practice, Patterns & Interviews 32

Cheat Sheets 34

START HERE

Before You Begin

What DSA is, what you need, and how to study so it sticks.

What are data structures and algorithms?

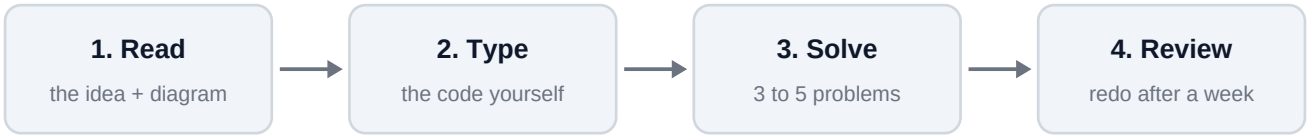
A **data structure** is a way of organising data in memory so that certain jobs are fast. An **algorithm** is a step-by-step recipe for solving a problem. Think of a library: the shelves and catalogue are the data structure, and "look up the call number, walk to the shelf, scan along it" is the algorithm.

Picking the right pair is the difference between a program that answers in a blink and one that never finishes. That is why DSA is the core of coding interviews and of writing software that scales.

What you need

- **A Mac with Xcode** (free): File > New > Playground, or a Command Line Tool project. The Swift Playgrounds app on Mac or iPad also works.
- **No Mac?** Use an online Swift compiler in your browser, or install Swift for Linux or Windows from swift.org.
- **Swift 5 or newer.** Every example here uses plain standard-library Swift, no packages.
- Some programming exposure helps, but Chapter 1 teaches the Swift you need from zero.

The study loop



Five rules that make it stick

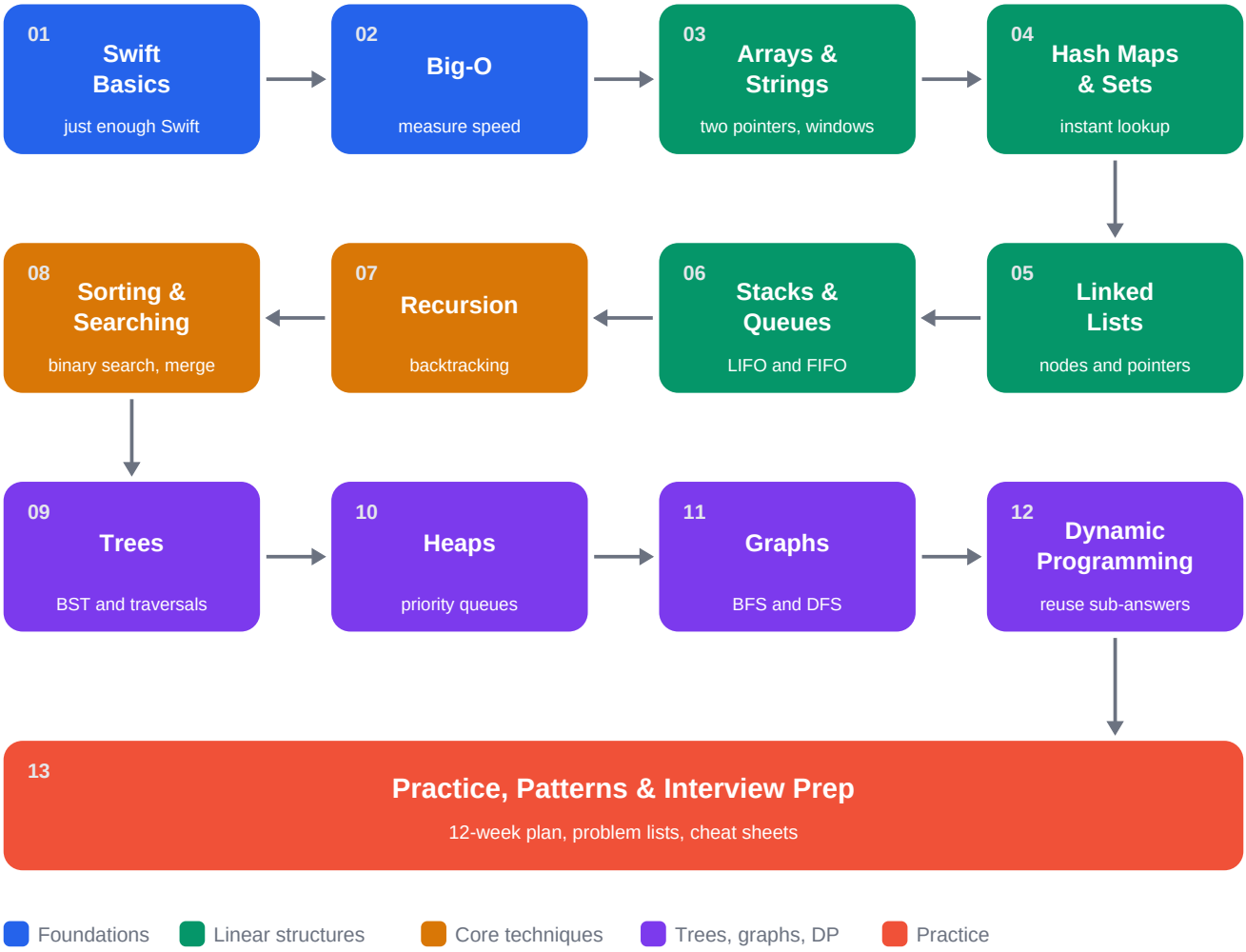
- **Type the code**, never paste it. Your fingers learn what your eyes skim past.
- **Predict the output** before you run anything. Wrong guesses are where learning happens.
- **Draw it on paper.** Arrays, nodes, trees: sketch every step of a small example.
- **State the complexity out loud** for every solution: time and space.
- **Redo problems after a week** without peeking. Real recall beats re-reading.

How each chapter works
Idea in plain words, then a diagram, then Swift code you can paste into a playground, then complexity, then a short "Try it" list. Each code block is self-contained, so give it its own playground page.

YOUR MAP

The Roadmap

Thirteen stops. Follow them in order; each one builds on the last.



Follow the snake from the top left. The first two stops give you the language and the yardstick (Big-O). The green stops are the "line-shaped" structures every other topic leans on. Amber stops are techniques you will reuse everywhere. Purple stops are where interview questions get interesting. The orange stop turns knowledge into skill.

> **Pace yourself**
> One chapter a week gives you a 12-week path (full table in Chapter 13). If a chapter feels easy, move on. If it feels hard, repeat it. Understanding beats speed.

CHAPTER 1

1. Swift Basics

Just enough Swift to follow everything that comes after.

Skip what you already know. Everything here shows up later, so skim even the parts you recognise. Paste each block into a playground and change values to see what happens.

1.1 Constants, variables, and types

```
let language = "Swift"      // let = constant (cannot change)
var score = 10              // var = variable (can change)
score += 5                  // now 15

let height: Double = 1.75   // explicit type
let isReady = true          // Bool
let letter: Character = "A"
print("Score is \(score)")  // string interpolation -> Score is 15
```

Swift infers types, so you rarely write them. The ones you will use constantly: `Int`, `Double`, `Bool`, `String`, `Character`.

1.2 Control flow

```
for i in 0..<3 { print(i) }        // 0 1 2   (up to, not including, 3)
for i in 1...3 { print(i) }        // 1 2 3   (inclusive)

var n = 3
while n > 0 {
    n -= 1
}

let x = 7
if x % 2 == 0 {
    print("even")
} else {
    print("odd")                   // prints odd
}
```

1.3 Functions

```
func add(_ a: Int, _ b: Int) -> Int {
    return a + b
}
print(add(2, 3))     // 5

// 'inout' lets a function change the caller's variable
func doubleIt(_ x: inout Int) { x *= 2 }
var v = 4
doubleIt(&v)
print(v)             // 8
```

The underscores mean "no argument label", so you call `add(2, 3)` instead of `add(a: 2, b: 3)`. Most DSA code uses this style.

1.4 Optionals: a value or nothing

```
var maybe: Int? = nil        // "Int?" = an Int, or nothing (nil)
maybe = 5

if let value = maybe {       // safe unwrap
    print(value)             // 5
}
let safe = maybe ?? 0        // default when nil

func firstEven(_ a: [Int]) -> Int? {
    for x in a where x % 2 == 0 { return x }
    return nil
}
print(firstEven([1, 3, 4]) ?? -1)   // 4
```

Optionals appear everywhere in DSA: "the node might not exist", "the key might be missing". Unwrap with `if let`, `guard let`, or `??`. Avoid the force-unwrap `!` unless you are certain.

1.5 Collections: Array, Dictionary, Set

```
var nums = [3, 1, 4]             // Array
nums.append(1)                   // [3, 1, 4, 1]
print(nums[0], nums.count)       // 3 4

var ages = ["Ava": 20, "Ben": 22]   // Dictionary [String: Int]
ages["Cy"] = 19
print(ages["Ava"] ?? -1)         // 20

var seen: Set<Int> = [1, 2]      // Set: unique values only
seen.insert(3)
print(seen.contains(3))          // true
```

1.6 Structs vs classes (important!)

```
struct Point { var x: Int; var y: Int }   // value type: copied on assignment
var p1 = Point(x: 1, y: 2)
var p2 = p1
p2.x = 99
print(p1.x)                      // 1   (p1 unchanged)

class Box { var value = 0 }               // reference type: shared
let b1 = Box()
let b2 = b1
b2.value = 99
print(b1.value)                  // 99  (same object!)

enum Direction { case up, down, left, right }
```

> **Why this matters**
> Linked lists and trees are built from **classes** so that nodes can point to the same object. Stacks, queues, and heaps are usually **structs** wrapping an array.

1.7 Closures and generics

```
let nums = [5, 2, 8]
let ascending = nums.sorted { $0 < $1 }      // [2, 5, 8]
let doubled = nums.map { $0 * 2 }            // [10, 4, 16]
let evens = nums.filter { $0 % 2 == 0 }      // [2, 8]
let total = nums.reduce(0, +)                // 15

// Generics: one function that works for any type T
func swapValues<T>(_ a: inout T, _ b: inout T) {
    let tmp = a
    a = b
    b = tmp
}
var s1 = "hi", s2 = "yo"
swapValues(&s1, &s2)
print(s1, s2)                                // yo hi
```

1.8 Swift gotchas in DSA code

- An out-of-range index (`a[10]` on a 3-element array) **crashes**. Check bounds first.
- Strings cannot be indexed with an Int. Convert once: `let chars = Array(s)`.
- Arithmetic overflow also crashes. Keep an eye on big products and sums.
- To change a parameter inside a function, mark it `inout` and pass it with `&`.
- Array slices like `a[2...]` keep the original indices. Wrap in `Array(...)` if you want a fresh zero-based array.

Try it

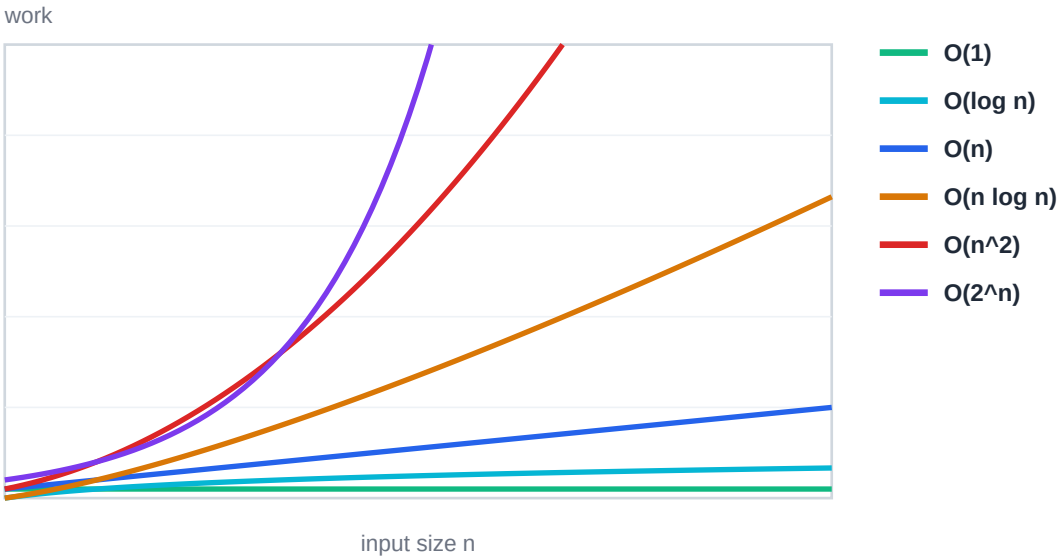
- Write `func isEven(_ n: Int) -> Bool` and test it on five numbers.
- Write a function that returns the largest value in an array, or `nil` if it is empty.
- Make a `struct` and a `class` with one property each. Copy both and mutate the copy. Explain the difference in your own words.

CHAPTER 2

2. Big-O Notation

A yardstick for how fast code slows down as the input grows.

Timing code in seconds depends on your laptop. **Big-O** ignores the machine and asks one question: *as the input size n grows, how fast does the amount of work grow?* Finding a name in a phone book by flipping page by page is O(n). Opening to the middle and halving each time is O(log n). Both find the name; one scales far better.



The steeper the curve, the sooner your program becomes unusable. At n = 1,000:

Complexity	Name	Rough steps at n = 1,000	Typical example
O(1)	constant	1	array index, dictionary lookup
O(log n)	logarithmic	~10	binary search
O(n)	linear	1,000	one loop over the data
O(n log n)	linearithmic	~10,000	good sorting (merge sort)
O(n^2)	quadratic	1,000,000	loop inside a loop
O(2^n)	exponential	astronomically large	trying every subset

Reading code for its complexity

```swift
// O(1): same work no matter how big the array is
func first(_ a: [Int]) -> Int? { a.first }

// O(n): touches each element once
func sum(_ a: [Int]) -> Int {
    var total = 0
    for x in a { total += x }
    return total
}

// O(n^2): a loop inside a loop
func hasPairSum(_ a: [Int], _ target: Int) -> Bool {
    for i in 0..<a.count {
        for j in (i + 1)..<a.count {
            if a[i] + a[j] == target { return true }
        }
    }
    return false
}

// O(log n): the problem halves every step
func countHalvings(_ n: Int) -> Int {
    var n = n, steps = 0
    while n > 1 { n /= 2; steps += 1 }
    return steps
}
```

Four rules of thumb

- **Drop constants:** 3n is O(n). We care about the shape, not the multiplier.
- **Drop smaller terms:** n^2 + n + 10 is O(n^2).
- **Sequential steps add, nested steps multiply:** two separate loops are O(n + n) = O(n), a loop inside a loop is O(n x n).
- **Halving the problem each step** is O(log n); **branching twice per level** is O(2^n).

Space complexity and other cases

Space complexity counts the extra memory your algorithm uses beyond the input. Building a copy of an array is O(n) space; swapping in place is O(1). Complexity can also differ by situation: **best case** (lucky input), **average case**, and **worst case**. Interviews usually mean worst case unless you say otherwise. **Amortized O(1)** means "O(1) on average over many operations", which is how appending to an array behaves.

Try it

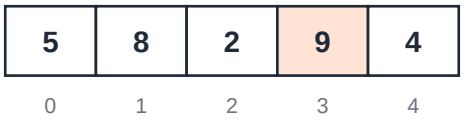
- What is the time complexity of a function with three sequential loops over n items? Of three nested loops?
- Why is `hasPairSum` above O(n^2)? What would make a faster version possible? (Hint: Chapter 4.)

CHAPTER 3

3. Arrays & Strings

The most common structure, and three techniques that unlock half of all problems.

An **array** stores items side by side in memory, each with a numbered position (index) starting at 0. Because the computer can jump straight to any slot, reading `a[3]` is instant. The cost comes when you insert or remove in the middle: everything after it has to shuffle over.



a[3] is 9. The index is the address; no searching needed.

Operation	Time	Note
Read or write `a[i]`	O(1)	direct jump
Search an unsorted array	O(n)	may check every element
Append at the end	O(1) amortized	occasionally the array grows and copies
Insert or remove at front or middle	O(n)	elements shift
Remove the last element	O(1)	

Technique 1: Two pointers

Keep one index at each end (or two indices moving at different speeds) and walk them toward each other. It turns many O(n^2) pair-checks into one O(n) pass.

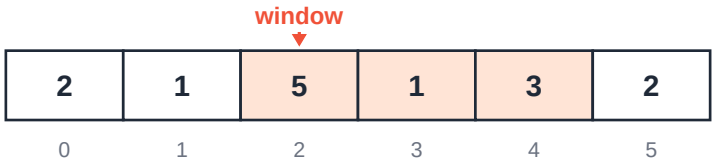



```
func reverse(_ a: inout [Int]) {
    var l = 0, r = a.count - 1
    while l < r {
        a.swapAt(l, r)
        l += 1
        r -= 1
    }
}
var xs = [1, 2, 3, 4, 5]
reverse(&xs)
print(xs)   // [5, 4, 3, 2, 1]        Time O(n), Space O(1)
```

```swift
func isPalindrome(_ s: String) -> Bool {
    let chars = Array(s.lowercased().filter { $0.isLetter || $0.isNumber })
    var l = 0, r = chars.count - 1
    while l < r {
        if chars[l] != chars[r] { return false }
        l += 1
        r -= 1
    }
    return true
}
print(isPalindrome("A man, a plan, a canal: Panama"))   // true
```

Technique 2: Sliding window

To examine every run of k neighbours, do not recompute from scratch. Slide the window one step: add the new element on the right, subtract the one that left.




```swift
func maxSum(_ a: [Int], _ k: Int) -> Int {
    guard k > 0, a.count >= k else { return 0 }
    var window = a[0..<k].reduce(0, +)
    var best = window
    for i in k..<a.count {
        window += a[i] - a[i - k]       // add right, drop left
        best = max(best, window)
    }
    return best
}
print(maxSum([2, 1, 5, 1, 3, 2], 3))    // 9   (5 + 1 + 3)    Time O(n)
```

Technique 3: Prefix sums

Precompute running totals once, then answer any "sum of a range" question in O(1).

```swift
func prefixSums(_ a: [Int]) -> [Int] {
    var p = [0]
    for x in a { p.append(p[p.count - 1] + x) }
    return p
}
let p = prefixSums([3, 1, 4, 1, 5])      // [0, 3, 4, 8, 9, 14]
print(p[4] - p[1])                       // sum of a[1..<4] = 1 + 4 + 1 = 6
```

> **Strings in Swift**
> A `String` is a sequence of `Character`s but cannot be indexed with an Int (Unicode makes that slow). For algorithm work, convert once with `Array(s)`, do your work on the array, and convert back with `String(chars)`.

Try it

- **Move Zeroes:** push all zeros to the end, keeping the other order (two pointers).

- **Best Time to Buy and Sell Stock:** one pass, track the lowest price so far.
- **Maximum Subarray:** find the contiguous run with the biggest sum.
- **Longest Substring Without Repeating Characters:** sliding window plus a set (needs Chapter 4).

CHAPTER 4

4. Hash Maps & Sets

Instant lookup: the single most useful trick in interviews.

A **hash map** (Swift: `Dictionary`) stores key-value pairs. A *hash function* turns each key into a number that says which "bucket" to use, like a coat-check where your ticket number tells the attendant exactly where your coat hangs. A **set** is the same idea storing only keys. Lookup, insert, and delete are **O(1) on average** (worst case O(n) if many keys collide, which is rare).

Structure	Lookup	Insert	Delete	Ordered?
`Dictionary`	O(1) avg	O(1) avg	O(1) avg	No
`Set`	O(1) avg	O(1) avg	O(1) avg	No
`Array` (for comparison)	O(n) search	O(1) end	O(n)	By index

Keys must be `Hashable` (Int, String, Character, and structs of those all are). Do not rely on iteration order: Swift does not promise one.

Pattern: "Have I seen this before?"

Two Sum: find two numbers adding to a target. Brute force checks every pair (O(n^2)). With a dictionary, for each number you ask "have I already seen the partner I need?"

```
func twoSum(_ nums: [Int], _ target: Int) -> [Int]? {
    var seen: [Int: Int] = [:]              // value -> index
    for (i, x) in nums.enumerated() {
        if let j = seen[target - x] { return [j, i] }
        seen[x] = i
    }
    return nil
}
print(twoSum([2, 7, 11, 15], 9) ?? [])      // [0, 1]      Time O(n), Space O(n)
```

Pattern: counting

```
func isAnagram(_ a: String, _ b: String) -> Bool {
    var count: [Character: Int] = [:]
    for ch in a { count[ch, default: 0] += 1 }
    for ch in b { count[ch, default: 0] -= 1 }
    return count.values.allSatisfy { $0 == 0 }
}
print(isAnagram("listen", "silent"))        // true
```

Pattern: detecting duplicates with a Set

```swift
func hasDuplicate(_ a: [Int]) -> Bool {
    var seen = Set<Int>()
    for x in a {
        if !seen.insert(x).inserted { return true }   // false if already present
    }
    return false
}
print(hasDuplicate([1, 2, 3, 1]))           // true
```

> **The trade**
> Hash maps usually trade **memory for speed**: O(n) extra space buys you O(1) lookups. Whenever a brute-force solution keeps searching for something, ask whether a dictionary can remember it instead.

Try it

- **Contains Duplicate**, **Valid Anagram**, **Group Anagrams** (key = sorted letters).
- **Top K Frequent Elements:** count with a dictionary, then sort or use a heap (Chapter 10).
- Rewrite `hasPairSum` from Chapter 2 in O(n) using a set.

CHAPTER 5

5. Linked Lists

Data as a chain of nodes, each pointing to the next.

A **linked list** is a chain of small objects called *nodes*. Each node holds a value and a pointer to the next node; the last points to `nil`. You only keep the `head` (first node) and follow the arrows. Unlike arrays, nodes can live anywhere in memory, so inserting at the front is instant, but reaching the 5th item means walking past four others.



Operation	Linked list	Array
Access the i-th item	O(n)	O(1)
Insert or remove at the front	O(1)	O(n)
Insert after a node you already hold	O(1)	O(n)
Search for a value	O(n)	O(n)

Building one

```
class ListNode {
    var val: Int
    var next: ListNode?
    init(_ val: Int, _ next: ListNode? = nil) {
        self.val = val
        self.next = next
    }
}
```

```swift
struct LinkedList {
    var head: ListNode?

    mutating func prepend(_ v: Int) {          // O(1)
        head = ListNode(v, head)
    }

    mutating func append(_ v: Int) {           // O(n): must walk to the end
        let node = ListNode(v)
        guard var cur = head else { head = node; return }
        while let nxt = cur.next { cur = nxt }
        cur.next = node
    }

    func toArray() -> [Int] {
        var out: [Int] = []
        var cur = head
        while let node = cur {
            out.append(node.val)
            cur = node.next
        }
        return out
    }
}
```

```swift
var list = LinkedList()
list.prepend(2)
list.prepend(1)
list.append(3)
print(list.toArray())      // [1, 2, 3]
```

Classic 1: Reverse a list

Walk once, flipping each arrow to point backwards. You need three names: **prev**, **cur**, and the saved **next**, so you never lose the rest of the chain.

```swift
func reverseList(_ head: ListNode?) -> ListNode? {
    var prev: ListNode? = nil
    var cur = head
    while let node = cur {
        let next = node.next      // 1. remember the rest
        node.next = prev          // 2. flip the arrow
        prev = node               // 3. step prev forward
        cur = next                // 4. step cur forward
    }
    return prev
}
list.head = reverseList(list.head)
print(list.toArray())      // [3, 2, 1]        Time O(n), Space O(1)
```

Classic 2: Fast and slow pointers (cycle detection)

Move one pointer one step and another two steps at a time. If the list loops, the fast one eventually laps the slow one. If it ends, the fast one hits `nil`. The same trick finds the middle of a list: when fast reaches the end, slow is halfway.

```swift
func hasCycle(_ head: ListNode?) -> Bool {
    var slow = head, fast = head
    while fast != nil && fast?.next != nil {
        slow = slow?.next
        fast = fast?.next?.next
        if slow === fast { return true }     // === compares object identity
    }
    return false
}
```

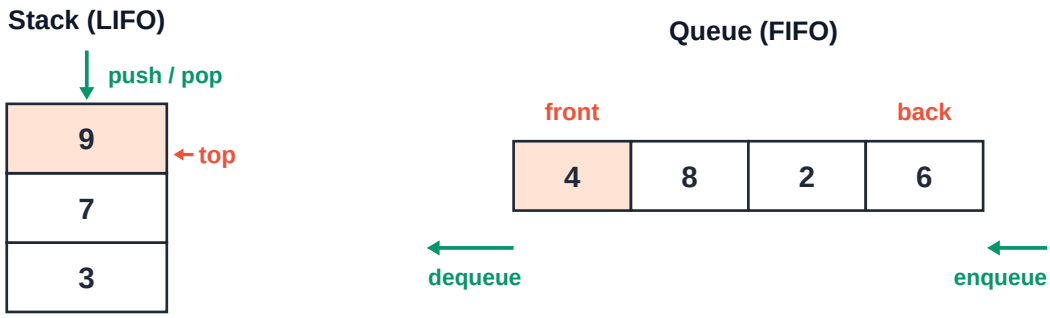
Try it

- **Merge Two Sorted Lists** and **Remove Nth Node From End** (hint: two pointers n apart).
- **Middle of the Linked List** with fast and slow pointers.
- Draw reverseList on paper for [1, 2, 3], redrawing the arrows after every loop pass.

CHAPTER 6

6. Stacks & Queues

Two simple rules for who goes next: last-in-first-out and first-in-first-out.



A **stack** is a pile of plates: you add and remove only at the top (LIFO). Browser back buttons, undo, and the function call stack all work this way. A **queue** is a line at a ticket counter: join at the back, leave from the front (FIFO). Task schedulers and breadth-first search use queues.

Swift has no built-in stack or queue type, but an array does the job. `append` and `popLast` are both O(1), which is exactly a stack.

A stack

```
struct Stack<T> {
    private var items: [T] = []
    var isEmpty: Bool { items.isEmpty }
    var top: T? { items.last }
    mutating func push(_ x: T) { items.append(x) }     // O(1)
    mutating func pop() -> T? { items.popLast() }      // O(1)
}
var s = Stack<Int>()
s.push(1); s.push(2); s.push(3)
print(s.pop() ?? -1, s.top ?? -1)    // 3 2
```

Stack problem: valid brackets

Push every opener. On a closer, the most recent opener must match, which is exactly what the top of a stack holds.

```
func isValid(_ s: String) -> Bool {
    let pairs: [Character: Character] = [")": "(", "]": "[", "}": "{"]
    var stack: [Character] = []
    for ch in s {
        if let open = pairs[ch] {                  // ch is a closer
            if stack.popLast() != open { return false }
        } else {
            stack.append(ch)                       // ch is an opener
        }
    }
    return stack.isEmpty
}
print(isValid("{[()]}"))    // true
print(isValid("(]"))        // false          Time O(n), Space O(n)
```

A queue

Beware: `Array.removeFirst()` is O(n) because everything shifts. Instead, keep a `head` index and move it forward on dequeue.

```
struct Queue<T> {
    private var items: [T] = []
    private var head = 0
    var isEmpty: Bool { head >= items.count }
    mutating func enqueue(_ x: T) { items.append(x) }          // O(1)
    mutating func dequeue() -> T? {                            // O(1)
        guard head < items.count else { return nil }
        let x = items[head]
        head += 1
        return x
    }
}
var q = Queue<String>()
q.enqueue("a"); q.enqueue("b")
print(q.dequeue() ?? "-")    // a   (first in, first out)
```

Going further
This queue never frees consumed slots, which is fine for learning and for most interview problems. Production code can use the **Deque** type from the open-source **swift-collections** package.

Try it

- **Min Stack:** a stack that also returns its minimum in O(1).
- **Daily Temperatures:** classic "monotonic stack" problem.
- **Implement Queue using Stacks:** two stacks, one for input and one for output.

CHAPTER 7

7. Recursion

A function that solves a problem by calling itself on a smaller piece.

Recursion is a function calling itself on a smaller version of its problem. Every recursive function needs two parts: a **base case** (the smallest problem, answered directly, so the calls stop) and a **recursive case** (shrink the problem and trust the call to solve it). Each call waits on the **call stack** until the one below it returns.

```
func factorial(_ n: Int) -> Int {
    if n <= 1 { return 1 }              // base case
    return n * factorial(n - 1)         // recursive case
}
print(factorial(5))                     // 120
// factorial(5) = 5 * factorial(4) = 5 * 4 * factorial(3) = ... = 120
```

Time O(n), Space O(n) because n calls sit on the stack at once. Forget the base case and the stack overflows and the program crashes.

When recursion repeats work: memoization

Naive Fibonacci recomputes the same values exponentially many times. Remember each answer in a dictionary and it collapses from O(2^n) to O(n). This idea is the seed of Chapter 12.

```
func fib(_ n: Int, _ memo: inout [Int: Int]) -> Int {
    if n < 2 { return n }
    if let cached = memo[n] { return cached }
    let result = fib(n - 1, &memo) + fib(n - 2, &memo)
    memo[n] = result
    return result
}
var memo: [Int: Int] = [:]
print(fib(40, &memo))                   // 102334155   (instant)
```

Backtracking: try, recurse, undo

For "generate all possibilities" problems, make a choice, recurse, then **undo** the choice so you can try the next one. Here: all subsets. For each number you either skip it or take it.

```
func subsets(_ nums: [Int]) -> [[Int]] {
    var result: [[Int]] = []
    var current: [Int] = []
    func go(_ i: Int) {
        if i == nums.count { result.append(current); return }
        go(i + 1)                       // choice 1: skip nums[i]
        current.append(nums[i])         // choice 2: take nums[i]
        go(i + 1)
        current.removeLast()            // undo
    }
    go(0)
    return result
}
print(subsets([1, 2]))     // [[], [2], [1], [1, 2]]       Time O(2^n)
```

A way to think
Do not trace every call in your head. Instead ask: *"If the smaller call magically works, what do I need to do to finish my own?"* Then check the base case.

Try it

- Write `sumDigits(_ n: Int)` recursively. Then `power(_ x: Int, _ n: Int)`.
- **Climbing Stairs** as plain recursion, then add memoization.
- **Permutations** and **Generate Parentheses** (backtracking).

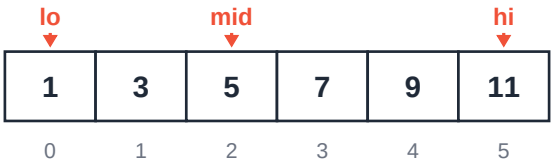
CHAPTER 8

8. Sorting & Searching

Binary search, and the sorting algorithms worth knowing.

Binary search

If the data is **sorted**, you never need to check everything. Look at the middle: if the target is smaller, discard the right half; if larger, discard the left half. Repeat. Each step halves the problem, so n = 1,000,000 takes about 20 steps.




```
func binarySearch(_ a: [Int], _ target: Int) -> Int? {
    var lo = 0, hi = a.count - 1
    while lo <= hi {
        let mid = lo + (hi - lo) / 2          // avoids overflow
        if a[mid] == target { return mid }
        if a[mid] < target { lo = mid + 1 } else { hi = mid - 1 }
    }
    return nil
}
print(binarySearch([1, 3, 5, 7, 9, 11], 7) ?? -1)    // 3     Time O(log n)
```

Insertion sort (simple, O(n^2))

Grow a sorted section on the left. Take the next item and slide it left until it fits. Good for tiny or nearly sorted data, and a clear way to learn how sorting works.

```
func insertionSort(_ a: inout [Int]) {
    guard a.count > 1 else { return }
    for i in 1..<a.count {
        let key = a[i]
        var j = i - 1
        while j >= 0 && a[j] > key {
            a[j + 1] = a[j]                   // shift bigger items right
            j -= 1
        }
        a[j + 1] = key
    }
}
var data = [5, 2, 4, 1]
insertionSort(&data)
print(data)                                   // [1, 2, 4, 5]
```

Merge sort (divide and conquer, O(n log n))

Split the array in half, sort each half recursively, then **merge** two sorted halves into one by always taking the smaller front item. Reliable and predictable.

```swift
func mergeSort(_ a: [Int]) -> [Int] {
    if a.count <= 1 { return a }
    let mid = a.count / 2
    let left = mergeSort(Array(a[..<mid]))
    let right = mergeSort(Array(a[mid...]))
    return merge(left, right)
}

func merge(_ l: [Int], _ r: [Int]) -> [Int] {
    var out: [Int] = []
    var i = 0, j = 0
    while i < l.count && j < r.count {
        if l[i] <= r[j] { out.append(l[i]); i += 1 }
        else { out.append(r[j]); j += 1 }
    }
    out += l[i...]                              // leftovers from either side
    out += r[j...]
    return out
}
print(mergeSort([38, 27, 43, 3, 9, 82, 10]))  // [3, 9, 10, 27, 38, 43, 82]
```

Quick sort (the idea)

Pick a pivot, split into smaller / equal / larger, and sort the outer groups recursively. This readable version uses extra memory; the in-place version is a good later exercise.

```swift
func quickSort(_ a: [Int]) -> [Int] {
    if a.count <= 1 { return a }
    let pivot = a[a.count / 2]
    let less = a.filter { $0 < pivot }
    let equal = a.filter { $0 == pivot }
    let greater = a.filter { $0 > pivot }
    return quickSort(less) + equal + quickSort(greater)
}
```

Algorithm	Best	Average	Worst	Extra space
Insertion sort	O(n)	O(n^2)	O(n^2)	O(1)
Merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick sort	O(n log n)	O(n log n)	O(n^2)	O(log n) in place
Swift `sort()` / `sorted()`	O(n log n)	O(n log n)	O(n log n)	varies

> **In real code, just call sorted()**
> Learn these to understand the ideas, but in projects and interviews that allow it, use the built-in `sorted()`. Sorting first often unlocks an O(n log n) solution to a problem that looks O(n^2).

Try it

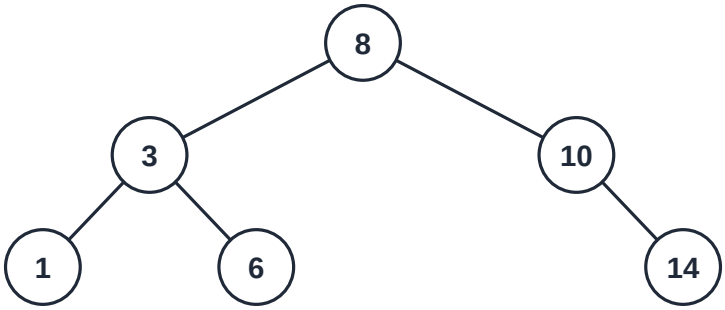
- **Binary Search**, then **Search in Rotated Sorted Array** (binary search with a twist).
- **Merge Intervals:** sort by start, then sweep.
- Trace mergeSort on [4, 1, 3, 2] by hand, drawing each split and merge.

CHAPTER 9

9. Trees

Hierarchies where every node has children: the shape behind folders, HTML, and search.

A **tree** is a linked structure where each node can have several children and exactly one parent (except the **root**). In a **binary tree** each node has at most two: `left` and `right`. A **binary search tree (BST)** adds one rule: everything in the left subtree is smaller than the node, everything in the right is larger. That rule lets you search by halving, like binary search.



Search for 6: start at 8, go left (6 < 8) to 3, go right (6 > 3) to 6. Found in 3 steps.

```
class TreeNode {
    var val: Int
    var left: TreeNode?
    var right: TreeNode?
    init(_ val: Int) { self.val = val }
}

func bstInsert(_ root: TreeNode?, _ v: Int) -> TreeNode {
    guard let root = root else { return TreeNode(v) }
    if v < root.val { root.left = bstInsert(root.left, v) }
    else { root.right = bstInsert(root.right, v) }
    return root
}

func bstContains(_ root: TreeNode?, _ v: Int) -> Bool {
    var cur = root
    while let n = cur {
        if v == n.val { return true }
        cur = v < n.val ? n.left : n.right
    }
    return false
}
```

BST insert and search are **O(h)** where h is the height: O(log n) for a balanced tree, but O(n) if the tree degenerates into a chain (e.g. inserting already sorted data).

Traversals: visiting every node

DFS goes deep first: *inorder* (left, node, right), *preorder* (node, left, right), *postorder* (left, right, node). **BFS** (level-order) visits row by row using a queue. Inorder on a BST gives the values in sorted order.

```swift
func inorder(_ n: TreeNode?, _ out: inout [Int]) {
    guard let n = n else { return }
    inorder(n.left, &out)
    out.append(n.val)
    inorder(n.right, &out)
}

func height(_ n: TreeNode?) -> Int {
    guard let n = n else { return 0 }
    return 1 + max(height(n.left), height(n.right))
}
```

```swift
func levelOrder(_ root: TreeNode?) -> [[Int]] {
    guard let root = root else { return [] }
    var result: [[Int]] = []
    var queue: [TreeNode] = [root]
    var head = 0
    while head < queue.count {
        let levelEnd = queue.count            // nodes in this row
        var level: [Int] = []
        while head < levelEnd {
            let node = queue[head]
            head += 1
            level.append(node.val)
            if let l = node.left { queue.append(l) }
            if let r = node.right { queue.append(r) }
        }
        result.append(level)
    }
    return result
}
```

```swift
var root: TreeNode? = nil
for v in [8, 3, 10, 1, 6, 14] { root = bstInsert(root, v) }

var out: [Int] = []
inorder(root, &out)
print(out)               // [1, 3, 6, 8, 10, 14]   (sorted!)
print(height(root))      // 3
print(levelOrder(root))  // [[8], [3, 10], [1, 6, 14]]
print(bstContains(root, 6), bstContains(root, 7))    // true false
```

Recipe for tree problems
Almost every tree question is solved by recursion with this shape: *handle nil (base case), solve the left, solve the right, combine the answers at this node.* `height` above is the template.

Try it

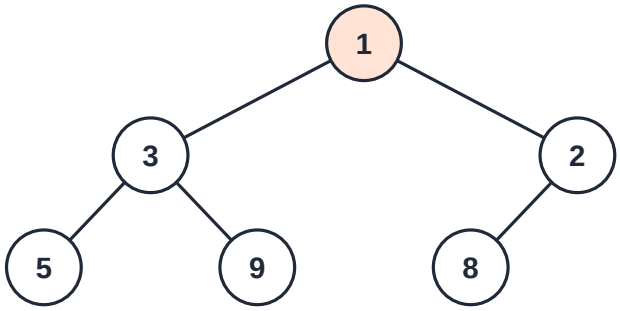
- **Maximum Depth of Binary Tree**, **Invert Binary Tree**, **Same Tree**.
- **Validate Binary Search Tree** (pass down allowed min and max).
- **Lowest Common Ancestor of a BST** and **Level Order Traversal** variants.

CHAPTER 10

10. Heaps & Priority Queues

Always know the smallest (or largest) item, cheaply.

A **priority queue** lets you add items and always pull out the one with the highest priority. It is usually built on a **heap**: a tree stored in an array where every parent is smaller than its children (a **min-heap**), so the minimum always sits at the top. Used for task schedulers, "top K" questions, and Dijkstra's shortest path.





The same heap as an array. For the item at index i: parent = (i - 1) / 2, left child = 2i + 1, right child = 2i + 2.

Operation	Time
Peek at the minimum	O(1)
Push (add an item)	O(log n): add at the end, bubble up
Pop (remove the minimum)	O(log n): move last to top, sink down
Build from n items	O(n)

```
struct MinHeap {
    private var a: [Int] = []
    var isEmpty: Bool { a.isEmpty }
    var peek: Int? { a.first }

    mutating func push(_ x: Int) {
        a.append(x)
        var i = a.count - 1
        while i > 0 {                          // bubble up
            let p = (i - 1) / 2
            if a[p] <= a[i] { break }
            a.swapAt(p, i)
            i = p
        }
    }

    mutating func pop() -> Int? {
        guard !a.isEmpty else { return nil }
        a.swapAt(0, a.count - 1)
```

```
        let top = a.removeLast()
        var i = 0
        while true {                            // sink down
            let l = 2 * i + 1, r = 2 * i + 2
            var smallest = i
            if l < a.count && a[l] < a[smallest] { smallest = l }
            if r < a.count && a[r] < a[smallest] { smallest = r }
            if smallest == i { break }
            a.swapAt(i, smallest)
            i = smallest
        }
        return top
    }
}
```

```
var h = MinHeap()
for x in [5, 3, 8, 1, 9, 2] { h.push(x) }
var ordered: [Int] = []
while let x = h.pop() { ordered.append(x) }
print(ordered)        // [1, 2, 3, 5, 8, 9]      (heap sort in 3 lines)
```

The "top K" trick
To find the K largest of n items, keep a min-heap of size K. For each new item, push it and pop if the size exceeds K. Time O(n log K), far better than sorting everything when K is small.

Try it

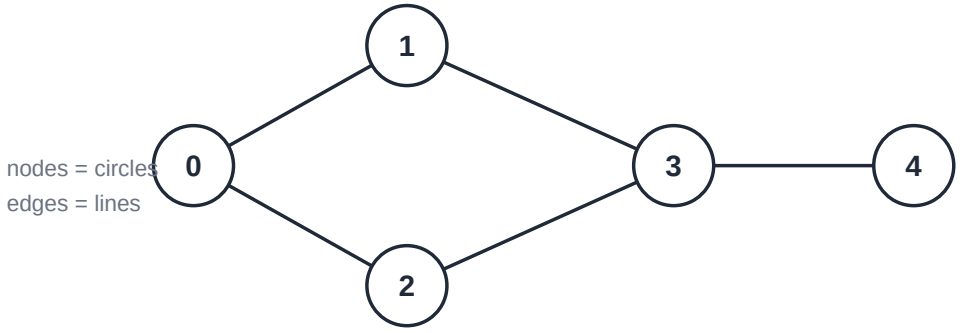
- **Kth Largest Element in an Array** and **Top K Frequent Elements**.
- **Merge k Sorted Lists** (heap of the current front of each list).
- Make the heap generic with a comparison closure so one type serves both min and max.

CHAPTER 11

11. Graphs

Things and the connections between them: maps, networks, dependencies.

A **graph** is a set of **nodes** (vertices) joined by **edges**. Roads between cities, friends on a social network, course prerequisites: all graphs. Edges can be directed or undirected, and may carry weights. The usual representation is an **adjacency list**: for each node, the list of its neighbours.




```
let graph: [[Int]] = [
    [1, 2],        // node 0 connects to 1 and 2
    [0, 3],        // node 1
    [0, 3],        // node 2
    [1, 2, 4],     // node 3
    [3]            // node 4
]
```

BFS: explore layer by layer

Use a **queue** and a **visited set**. Visit the start, then everything one step away, then two steps away. BFS finds the **shortest path in an unweighted graph**.

```
func bfs(_ g: [[Int]], _ start: Int) -> [Int] {
    var visited: Set<Int> = [start]
    var queue = [start]
    var head = 0
    var order: [Int] = []
    while head < queue.count {
        let node = queue[head]
        head += 1
        order.append(node)
        for nb in g[node] where !visited.contains(nb) {
            visited.insert(nb)
            queue.append(nb)
        }
    }
    return order
}
print(bfs(graph, 0))     // [0, 1, 2, 3, 4]
```

DFS: go as deep as possible, then back up

Use recursion (the call stack acts as the stack). Mark each node visited before exploring its neighbours so you never loop forever.

```
func dfs(_ g: [[Int]], _ node: Int,
         _ visited: inout Set<Int>, _ order: inout [Int]) {
    visited.insert(node)
    order.append(node)
    for nb in g[node] where !visited.contains(nb) {
        dfs(g, nb, &visited, &order)
    }
}
var seen = Set<Int>()
var order: [Int] = []
dfs(graph, 0, &seen, &order)
print(order)             // [0, 1, 3, 2, 4]
```

	BFS	DFS
Data structure	Queue	Stack / recursion
Explores	level by level	one path to the end first
Great for	shortest path (unweighted), nearest X	all paths, connectivity, cycles, backtracking
Time / Space	O(V + E) / O(V)	O(V + E) / O(V)

V = number of nodes, E = number of edges. A **grid** (maze, map of islands) is a graph in disguise: each cell is a node whose neighbours are up, down, left, and right.

Try it

- **Number of Islands** (DFS or BFS on a grid), **Flood Fill**.
- **Clone Graph** and **Course Schedule** (cycle detection / topological sort).
- Modify `bfs` to return the shortest distance from the start to every node.
- Next topics: Dijkstra (graphs + heap), union-find, topological sort.

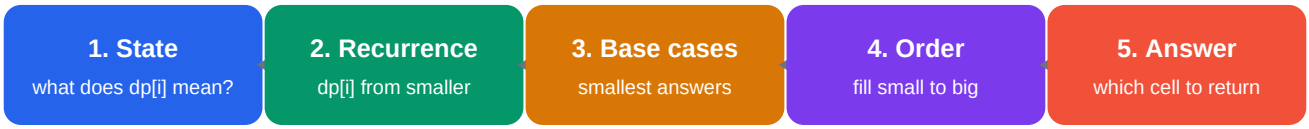
CHAPTER 12

12. Dynamic Programming

Solve a big problem by solving small ones once and reusing the answers.

Dynamic programming (DP) applies when a problem has *overlapping subproblems* (the same smaller question appears again and again) and *optimal substructure* (the best answer is built from best answers to smaller pieces). You saw the idea with memoized Fibonacci. Two styles: **top-down** (recursion plus a cache) and **bottom-up** (fill a table from the smallest case upward).

The five-step recipe



Example 1: Climbing Stairs

You can climb 1 or 2 steps at a time. How many ways to reach step n? The last move came from step n-1 or n-2, so **ways(n) = ways(n-1) + ways(n-2)**. Only the last two values matter, so we keep just two variables (O(1) space).

```
func climbStairs(_ n: Int) -> Int {
    if n <= 2 { return n }
    var a = 1, b = 2                  // ways for step 1 and step 2
    for _ in 3...n {
        (a, b) = (b, a + b)
    }
    return b
}
print(climbStairs(5))                 // 8          Time O(n), Space O(1)
```

Example 2: Coin Change (fewest coins)

State: `dp[a]` = fewest coins that make amount a. For each coin c that fits: `dp[a] = min(dp[a], dp[a - c] + 1)`. Base: `dp[0] = 0`.

```
func coinChange(_ coins: [Int], _ amount: Int) -> Int {
    if amount == 0 { return 0 }
    var dp = [Int](repeating: Int.max, count: amount + 1)
    dp[0] = 0
    for a in 1...amount {
        for c in coins where c <= a && dp[a - c] != Int.max {
            dp[a] = min(dp[a], dp[a - c] + 1)
        }
    }
    return dp[amount] == Int.max ? -1 : dp[amount]
}
print(coinChange([1, 2, 5], 11))      // 3   (5 + 5 + 1)      Time O(amount x coins)
```

How to spot a DP problem
Words like *"minimum"*, *"maximum"*, *"number of ways"*, or *"is it possible"* combined with choices at each step. First write the plain recursive solution, notice the repeated calls, then add a cache. Converting to a table is the optional last step.

Try it

- **House Robber** (like stairs, with a twist), **Maximum Subarray** as DP (Kadane).
- **Longest Common Subsequence** (a 2D table) and **Unique Paths** on a grid.
- Solve Climbing Stairs three ways: plain recursion, memoized, bottom-up. Compare speed at n = 40.

CHAPTER 13

13. Practice, Patterns & Interviews

Turn what you know into skill you can use under pressure.

The 12-week plan

About an hour a day. Each week: two days learning and typing the chapter code, then daily problems from that chapter, then one review session.

Week	Focus	Goal by Sunday
1	Ch 1: Swift basics	write small functions without looking anything up
2	Ch 2 + 3: Big-O, arrays & strings	5 two-pointer or window problems
3	Ch 4: hash maps & sets	5 problems, each with a complexity statement
4	Ch 5: linked lists	reverse a list from memory
5	Ch 6: stacks & queues	valid brackets + one monotonic stack problem
6	Ch 7: recursion	subsets and permutations from memory
7	Ch 8: sorting & searching	binary search without bugs, merge sort from memory
8	Ch 9: trees	6 tree problems using the recursion recipe
9	Ch 10: heaps	top-K problems, write MinHeap
10	Ch 11: graphs	BFS and DFS on a grid
11	Ch 12: dynamic programming	4 classic DP problems
12	Mixed review, mock interviews	solve unseen problems in 30 minutes

Pattern cheat sheet: which tool for which clue?

If you see...	Think...
sorted array, find a pair or range	two pointers, binary search
contiguous subarray or substring	sliding window, prefix sums
"have I seen this?", counting, matching	hash map or set
linked list, middle, cycle	fast and slow pointers
matching pairs, "next greater element", undo	stack
shortest path, level by level	BFS with a queue
all paths, connected regions, tree questions	DFS / recursion
all combinations or arrangements	backtracking
top K, repeatedly need the min or max	heap
min, max, or number of ways with choices	dynamic programming

Problem list by topic

Search these titles on LeetCode, NeetCode, or HackerRank. Do the first two or three in each row when you finish the chapter; return to the rest during review weeks. Popular curated lists such as the Blind 75 and NeetCode 150 cover these same patterns.

Topic	Problems
Arrays & strings	Two Sum II, Best Time to Buy and Sell Stock, Move Zeroes, Maximum Subarray, Container With Most Water, Valid Palindrome
Hashing	Contains Duplicate, Valid Anagram, Group Anagrams, Top K Frequent Elements, Longest Substring Without Repeating Characters
Linked lists	Reverse Linked List, Merge Two Sorted Lists, Linked List Cycle, Remove Nth Node From End
Stacks & queues	Valid Parentheses, Min Stack, Daily Temperatures, Implement Queue using Stacks
Recursion	Subsets, Permutations, Generate Parentheses, Combination Sum
Sorting & searching	Binary Search, Search in Rotated Sorted Array, Merge Intervals, Sort Colors
Trees	Maximum Depth of Binary Tree, Invert Binary Tree, Validate BST, Level Order Traversal, Lowest Common Ancestor
Heaps	Kth Largest Element in an Array, Last Stone Weight, Merge k Sorted Lists
Graphs	Number of Islands, Clone Graph, Course Schedule, Rotting Oranges
Dynamic programming	Climbing Stairs, House Robber, Coin Change, Longest Common Subsequence

During an interview: a repeatable routine

- **Restate** the problem and ask about edge cases (empty input, duplicates, negatives, size).
- **Work a small example** by hand and say what you notice.
- **Start with brute force** out loud and state its complexity, then improve it using a pattern.
- **Code cleanly**, narrate as you go, then **test** with your example and one edge case.
- **State final time and space complexity.**

QUICK REFERENCE

Cheat Sheets

Keep this page open while you practise.

Data structure operations

Structure	Access	Search	Insert	Delete	Notes
Array	O(1)	O(n)	O(n) / O(1) end	O(n)	contiguous, ordered
Linked list	O(n)	O(n)	O(1) at head	O(1) with node	pointer chain
Stack	-	-	O(1) push	O(1) pop	LIFO
Queue	-	-	O(1) enqueue	O(1) dequeue	FIFO
Hash map / set	-	O(1) avg	O(1) avg	O(1) avg	unordered
BST (balanced)	-	O(log n)	O(log n)	O(log n)	sorted order via inorder
BST (degenerate)	-	O(n)	O(n)	O(n)	a chain, avoid
Min-heap	O(1) min	O(n)	O(log n)	O(log n) min	priority queue
Graph (adj. list)	-	O(V + E)	O(1) edge	O(deg)	BFS / DFS O(V + E)

Swift standard-library quick reference

Need	Swift
Array append / pop last	`a.append(x)` `a.popLast()`
Swap two items	`a.swapAt(i, j)`
Sort (ascending / custom)	`a.sorted()` `a.sorted { $0.1 > $1.1 }`
Index and value together	`for (i, x) in a.enumerated() { }`
Dictionary with default	`d[key, default: 0] += 1`
Set insert / membership	`s.insert(x)` `s.contains(x)`
Min / max / sum	`a.min()` `a.max()` `a.reduce(0, +)`
Array of n zeros	`[Int](repeating: 0, count: n)`
2D grid of zeros	`[[Int]](repeating: [Int](repeating: 0, count: m), count: n)`
Largest / smallest Int	`Int.max` `Int.min`
String to characters and back	`Array(s)` `String(chars)`

Where to go next

- **The Swift Programming Language**, the free official book at docs.swift.org, to deepen the language.
- **Swift Algorithm Club** (open source on GitHub): many more structures and algorithms in Swift.

- **Grokking Algorithms** for friendly illustrated intuition; **Cracking the Coding Interview** for interview practice; **CLRS** when you want the full theory.
- Later topics: tries, union-find, Dijkstra, topological sort, segment trees, bit manipulation.

Last word
Nobody learns DSA by reading. You learn it by getting problems wrong, drawing the state, and trying again. Keep the loop going, one chapter at a time.